

**UNIVERSIDAD DEL VALLE DE GUATEMALA**

**PROGRAMACION ORIENTADA A OBJETOS**

**DIEGO ISAIAS MARROQUIN MOSCOSO**

## **LABORATORIO #2**

### **Análisis**

Responda las siguientes preguntas:

#### **1. ¿Qué propiedades y métodos tendrá cada clase?**

Se usarán cuatro clases: Ubicacion, Pista, Caso y Main. Ubicacion tendrá codigo, nombre, direccion, nivelRiesgo y estado. Tendrá un constructor, getters, setters para cambiar el riesgo y el estado, y toString() para mostrar sus datos.

Pista tendrá codigo, descripcion, tipoEvidencia, nivelImportante y nivelConfiable. Sus métodos serán el constructor, los getters, los setters de los datos que se pueden modificar y toString().

Caso tendrá nombre, codigo, detective, ubicaciones y pistas. Se encargará de registrar, consultar, modificar y eliminar los objetos de ambas estructuras. También tendrá métodos para contar los registros, calcular el promedio de importancia y encontrar la ubicación más peligrosa y las pistas con mayor importancia y confiabilidad.

Main tendrá main(), que mostrará las trece opciones del menú, pedirá los datos con Scanner y llamará a los métodos de Caso. No necesita atributos de instancia.

## **2. ¿Qué tipo deben tener las propiedades y métodos de cada clase?**

En Ubicacion, codigo, nombre, direccion y estado serán String; nivelRiesgo será int. En Pista, codigo, descripcion y tipoEvidencia serán String; nivelImportante y nivelConfiable serán int. Los getters devolverán el tipo del atributo, los setters serán void y toString() devolverá String.

En Caso, nombre, codigo y detective serán String. ubicaciones será Ubicacion[] y pistas será ArrayList<Pista>. Los métodos de registro, modificación y eliminación devolverán boolean para indicar si se realizó la operación. Las búsquedas devolverán Ubicacion o Pista, y obtenerPistas() devolverá ArrayList<Pista>.

Los conteos devolverán int y promedioNivelImportancia() devolverá double. mostrarUbicaciones() y mostrarPistas() devolverán boolean para indicar si había registros que mostrar. Los constructores no tienen tipo de retorno. En Main, main() será public static void y recibirá String[] args.

## **3. ¿Cuál de las propiedades identificadas debe implementarse utilizando un arreglo básico?**

### **¿Qué tipo de objetos almacenará y cuál será su tamaño?**

La propiedad ubicaciones de Caso será un arreglo básico de objetos Ubicacion. Se creará con new Ubicacion[5], porque cada caso puede tener como máximo cinco ubicaciones. Al principio sus posiciones tendrán null. El usuario elegirá una posición de 1 a 5 y el programa restará uno para trabajar con los índices de 0 a 4.

**4. ¿Cuál de las propiedades identificadas debe implementarse utilizando un ArrayList? ¿Qué tipo de objetos almacenará?**

La propiedad pistas de Caso será un ArrayList<Pista>. Guardará los objetos Pista completos y se creará vacío en el constructor. Así se podrán agregar o eliminar pistas sin definir desde el inicio cuántas habrá. Se utilizará ArrayList directamente, como lo pide el laboratorio.

**5. ¿Cuáles deben ser los modificadores de visibilidad de los miembros en cada clase?**

Los atributos de Caso, Ubicacion y Pista serán private para que no se cambien directamente desde Main. Los constructores y los métodos que se necesiten desde otras clases serán public. Los getters permitirán consultar los datos y los setters modificarán únicamente los atributos que corresponde cambiar. main() será public static void para que Java pueda iniciar el programa.

**6. ¿Qué parámetros serán requeridos por los métodos en sus clases?**

Ubicacion recibirá en su constructor codigo, nombre, direccion y estado de tipo String, y nivelRiesgo de tipo int. Pista recibirá codigo, descripcion y tipoEvidencia como String, y nivelImportante y nivelConfiable como int. Caso recibirá nombre, codigo y detective como String.

Para registrar una ubicación se necesitarán la posicion de tipo int y el objeto Ubicacion. Para obtenerla o descartarla bastará con la posicion. modificarUbicacion() recibirá la posicion, el nuevo riesgo de tipo int y el estado de tipo String.

registrarPista() recibirá un objeto Pista. obtenerPista() y eliminarPista() recibirán su código. modificarPista() recibirá código, descripción y tipoEvidencia como String, e importancia y confiabilidad como int.

Cada setter recibirá el nuevo valor de su atributo. Los getters, los conteos, los listados y los métodos del reporte no necesitarán parámetros. main() recibirá String[] args. Las firmas completas se detallan en la descripción de cada clase.

## **7. ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores deberán validarse antes de modificar el estado de los objetos?**

Los objetos recibirán sus datos por medio de los constructores. Al crear un Caso se guardarán su nombre, código y detective, se inicializará el arreglo con cinco posiciones null y se creará un ArrayList vacío. En Main, casoActual empezará en null, programaAbierto en true, validar en false y opcion en 0.

Antes de guardar o modificar se revisará que los textos no estén vacíos, que el riesgo y la importancia estén entre 1 y 10, y que la confiabilidad esté entre 0 y 100. También se comprobará que la posición exista, que esté libre al registrar y que contenga una ubicación al modificar o descartar. Para las pistas se verificará que el código no esté repetido y que exista antes de modificar o eliminar.

Los cambios se harán después de validar todos los datos. Si se escribe cancelar en un campo de texto o -1 en un campo numérico de una operación, se regresará al menú sin guardar los cambios de esa operación.

#### **8. ¿Cómo determinará si una posición del arreglo contiene una ubicación o contiene null?**

Primero se comprobará que el índice esté entre 0 y 4. Después se comparará `ubicaciones[posicion]` con `null`. Si es `null`, la posición está vacía; si no lo es, se puede consultar el objeto. `obtenerUbicacion()` devolverá `null` cuando no haya una ubicación o el índice sea inválido. Al recorrer el arreglo se omitirán las posiciones vacías y se mostrarán los datos de las ocupadas junto con su posición.

#### **9. ¿Cómo realizará las operaciones de búsqueda, modificación y eliminación dentro del ArrayList?**

`obtenerPista(codigo)` recorrerá el `ArrayList` con un `for-each` y comparará el código recibido con `getCodigo()` usando `equals()`. Si encuentra la pista la devolverá; si no, devolverá `null`. Esta búsqueda también servirá para evitar códigos repetidos al registrar.

Para modificar se buscará primero la pista y se usarán sus setters después de validar los nuevos datos. Para eliminar, se buscará el objeto y se quitará con `pistas.remove(pista)`. La eliminación se hará después de terminar la búsqueda. Si no existe la pista, el método devolverá `false` y `Main` informará al usuario.

#### **10. ¿Qué situaciones del programa pueden producir excepciones? Identifique qué excepciones deberán manejarse y en qué partes del programa utilizará try-catch y finally.**

`Scanner` puede generar `InputMismatchException` si se escribe texto, un decimal o un número demasiado grande cuando se espera un `int`. Se usará `try-catch` en el menú y en las lecturas numéricas. Al capturar el error se descartará la entrada con `nextLine()` para permitir otro intento.

Se usará `IllegalArgumentException` cuando el riesgo, la importancia o la confiabilidad estén fuera de sus rangos. También se controlarán las posiciones inválidas. Main usa `IllegalStateException` para una posición ocupada, `NullPointerException` de forma controlada cuando no encuentra un registro y `Exception` cuando un texto está vacío. Revisar los límites y los valores null antes de usar los objetos evita accesos incorrectos.

Para finally, usaré el cierre del Scanner al terminar el programa: el ciclo principal quedará dentro de un try y `scanner.close()` dentro de finally. Así se liberará el recurso tanto al salir normalmente como si ocurre una excepción. No se debe cerrar en cada intento de lectura, porque todavía se necesita para las siguientes opciones.

### **Requisitos funcionales del sistema:**

- Solicitar el nombre del caso, su código y el nombre del detective responsable antes de registrar ubicaciones o pistas.
- Permitir crear un nuevo caso que reemplace al anterior y comience con ambas estructuras vacías.
- Guardar un máximo de cinco ubicaciones en un arreglo básico de objetos Ubicacion.
- Registrar cada ubicación con código, nombre, dirección o descripción del lugar, nivel de riesgo y estado.
- Validar que el riesgo esté entre 1 y 10 al registrar o modificar. Un valor fuera del rango debe generar `IllegalArgumentException`.
- Solicitar la posición de la nueva ubicación y comprobar que esté dentro del arreglo, disponible y que los datos sean válidos.
- Mantener null en las posiciones vacías y comprobarlo antes de usar los métodos de una ubicación.
- Consultar todas las ubicaciones registradas y mostrar la posición que ocupa cada una, sin presentar los espacios vacíos como registros.
- Consultar una ubicación por su posición e informar si la posición es inválida o está vacía.
- Modificar el riesgo y el estado de una ubicación existente después de validar los nuevos datos.
- Descartar una ubicación y dejar nuevamente null en su posición.
- Almacenar las pistas en un `ArrayList<Pista>`, sin utilizar `List` ni establecer un tamaño fijo.
- Registrar cada pista con código, descripción, tipo de evidencia, nivel de importancia y nivel de confiabilidad.
- Validar una importancia de 1 a 10 y una confiabilidad de 0 a 100. Los valores fuera de estos rangos deben generar `IllegalArgumentException`.

- Evitar el registro de dos pistas con el mismo código.
- Consultar todas las pistas y mostrar un mensaje cuando el ArrayList esté vacío.
- Buscar una pista por su código recorriendo el ArrayList.
- Modificar los datos de una pista existente con las mismas validaciones del registro.
- Eliminar del ArrayList una pista localizada por su código.
- Mostrar la cantidad de ubicaciones registradas, los espacios disponibles y la ubicación con mayor riesgo.
- Mostrar la cantidad de pistas, la de mayor importancia, la de mayor confiabilidad y el promedio de importancia.
- Calcular los reportes usando solo los objetos registrados. Si no hay pistas, evitar la división entre cero y las búsquedas de máximos sobre objetos inexistentes.
- Manejar InputMismatchException, limpiar la entrada incorrecta y permitir que el usuario vuelva a intentar.
- Usar try-catch en las lecturas y validaciones, y al menos un finally para una acción final que deba ejecutarse aunque ocurra un error.
- Mantener un menú de trece opciones: nuevo caso; registrar, consultar, consultar una, modificar y descartar ubicación; registrar, consultar, buscar, modificar y eliminar pista; mostrar reporte; salir.
- Continuar funcionando después de errores controlados. Como opción adicional, permitir cancelar operaciones y volver al menú con una sola lectura de Enter.

## **Ubicacion**

```
- codigo: String

- nombre: String

- direccion: String

- nivelRiesgo: int

- estado: String

+ Ubicacion(codigo: String, nombre: String, direccion: String, nivelRiesgo: int, estado: String)

+ getCodigo(): String

+ getNombre(): String

+ getDireccion(): String

+ getNivelRiesgo(): int

+ getEstado(): String

+ setNivelRiesgo(nivel: int): void

+ setEstado(estado: String): void

+ toString(): String
```

### **Propósito de la clase:**

Representa uno de los lugares que se investigarán. Guarda sus datos y permite cambiar el riesgo y el estado conforme avance el caso.

### **Propósito de los atributos:**

- codigo: identifica la ubicación.
- nombre: guarda el nombre del lugar.

- `direccion`: indica dónde se encuentra o cómo se describe el lugar.
- `nivelRiesgo`: indica el riesgo de investigar el lugar, de 1 a 10.
- `estado`: guarda la situación actual de la ubicación, por ejemplo, pendiente o revisada.

**Propósito de los métodos:**

`Ubicacion(codigo, nombre, direccion, nivelRiesgo, estado)`: recibe código, nombre, dirección, riesgo y estado, y los asigna a los atributos.

`getCodigo()`: devuelve el valor de código.

`getNombre()`: devuelve el valor de nombre.

`getDireccion()`: devuelve el valor de dirección.

`getNivelRiesgo()`: devuelve el valor de nivelRiesgo.

`getEstado()`: devuelve el valor de estado.

`setNivelRiesgo(nivel)`: actualiza nivelRiesgo con el valor recibido.

`setEstado(estado)`: actualiza estado con el valor recibido.

`toString()`: devuelve un String con los datos de la ubicación para mostrarlos en consola.

## **Pista**

- codigo: String

- descripcion: String

- tipoEvidencia: String

- nivelImportante: int

- nivelConfiable: int

+ Pista(codigo: String, descripcion: String, tipoEvidencia: String, nivelImportante: int, nivelConfiable: int)

+ getCodigo(): String

+ getDescripcion(): String

+ getTipoEvidencia(): String

+ getNivelImportante(): int

+ getNivelConfiable(): int

+ setDescripcion(descripcion: String): void

+ setTipoEvidencia(tipoEvidencia: String): void

+ setNivelImportante(nivelImportante: int): void

+ setNivelConfiable(nivelConfiable: int): void

+ toString(): String

## **Propósito de la clase:**

Representa una evidencia encontrada durante la investigación. Guarda su descripción y los niveles que se usarán para comparar las pistas y preparar el reporte.

### **Propósito de los atributos:**

- `codigo`: identifica la pista y permite buscarla sin repetir códigos.
- `descripcion`: explica qué se encontró.
- `tipoEvidencia`: indica la clase de evidencia.
- `nivelImportante`: guarda la importancia de la pista, de 1 a 10, y se usa para obtener el máximo y el promedio.
- `nivelConfiable`: guarda la confiabilidad de la pista; el rango solicitado es de 0 a 100.

### **Propósito de los métodos:**

`Pista(codigo, descripcion, tipoEvidencia, nivelImportante, nivelConfiable)`: recibe código, descripción, tipo de evidencia, importancia y confiabilidad, y los guarda en el objeto.

`getCodigo()`: devuelve el valor de `codigo`.

`getDescripcion()`: devuelve el valor de `descripcion`.

`getTipoEvidencia()`: devuelve el valor de `tipoEvidencia`.

`getNivelImportante()`: devuelve el valor de `nivelImportante`.

`getNivelConfiable()`: devuelve el valor de `nivelConfiable`.

`setDescripcion(descripcion)`: actualiza `descripcion` con el valor recibido.

`setTipoEvidencia(tipoEvidencia)`: actualiza `tipoEvidencia` con el valor recibido.

`setNivelImportante(nivelImportante)`: actualiza `nivelImportante` con el valor recibido.

`setNivelConfiable(nivelConfiable)`: actualiza `nivelConfiable` con el valor recibido.

`toString()`: devuelve un `String` con los datos de la pista para mostrarlos en consola.

## Caso

- nombre: String
- codigo: String
- detective: String
- ubicaciones: Ubicacion[]
- pistas: ArrayList<Pista>
- + Caso(nombre: String, codigo: String, detective: String)
- + getNombre(): String
- + getCodigo(): String
- + getDetective(): String
- + registrarUbicacion(posicion: int, ubicacion: Ubicacion): boolean
- + obtenerUbicacion(posicion: int): Ubicacion
- + modificarUbicacion(posicion: int, riesgo: int, estado: String): boolean
- + descartarUbicacion(posicion: int): boolean
- + registrarPista(pista: Pista): boolean
- + obtenerPista(codigo: String): Pista
- + modificarPista(codigo: String, descripcion: String, tipoEvidencia: String, importancia: int, confiabilidad: int): boolean
- + eliminarPista(codigo: String): boolean
- + obtenerPistas(): ArrayList<Pista>
- + cantidadUbicacionesRegistradas(): int

+ espaciosDisponibles(): int

+ ubicacionMasPeligrosa(): Ubicacion

+ cantidadPistasRegistradas(): int

+ pistaMasImportante(): Pista

+ pistaMasConfiable(): Pista

+ promedioNivelImportancia(): double

+ mostrarPistas(): boolean

+ mostrarUbicaciones(): boolean

+ cantidadDeEspaciosDeUbicaciones(): int

**Propósito de la clase:**

Organiza la información de una investigación. Guarda sus datos generales, administra el arreglo de ubicaciones y el ArrayList de pistas, y realiza los cálculos del reporte.

**Propósito de los atributos:**

- nombre: identifica el caso por un nombre descriptivo.
- codigo: guarda el código de identificación del caso.
- detective: guarda el nombre del responsable de la investigación.
- ubicaciones: arreglo Ubicacion[] de cinco espacios. Cada posición contiene un objeto o null si está disponible.
- pistas: ArrayList<Pista> que aumenta o disminuye conforme se registran o eliminan evidencias.

### **Propósito de los métodos:**

`Caso(nombre, codigo, detective)`: recibe nombre, código y detective. Crea el arreglo de cinco ubicaciones y el `ArrayList` de pistas vacío.

`getNombre()`: devuelve el valor de nombre.

`getCodigo()`: devuelve el valor de codigo.

`getDetective()`: devuelve el valor de detective.

`registrarUbicacion(posicion, ubicacion)`: recibe un índice y una ubicación. Si el índice es válido y está libre, guarda el objeto y devuelve `true`; de lo contrario devuelve `false`.

`obtenerUbicacion(posicion)`: recibe un índice y devuelve la ubicación guardada. Devuelve `null` si el índice es inválido o el espacio está vacío.

`modificarUbicacion(posicion, riesgo, estado)`: recibe un índice, riesgo y estado. Actualiza una ubicación existente y devuelve `true`; si no existe o el índice es inválido devuelve `false`.

`descartarUbicacion(posicion)`: recibe un índice y deja `null` en una posición ocupada. Devuelve `true` si la descartó y `false` si no pudo hacerlo.

`registrarPista(pista)`: recibe una pista, comprueba que no sea `null` y que no exista su código, y la agrega con `add()`. Devuelve `true` si se registró o `false` si fue rechazada.

`obtenerPista(codigo)`: recibe un código, recorre el `ArrayList` y devuelve la pista que coincide. Si no hay coincidencia devuelve `null`.

`modificarPista(codigo, descripcion, tipoEvidencia, importancia, confiabilidad)`: recibe el código y los nuevos datos. Busca la pista y actualiza descripción, tipo, importancia y confiabilidad. Devuelve `true` si existe o `false` si no la encuentra.

`eliminarPista(codigo)`: recibe un código, busca la pista y la elimina con `remove()`. Devuelve `true` si se eliminó y `false` si no existe.

`obtenerPistas()`: no recibe parámetros y devuelve el `ArrayList` de pistas, incluso cuando está vacío.

`cantidadUbicacionesRegistradas()`: recorre el arreglo y cuenta los elementos distintos de `null`.

Devuelve la cantidad como `int`.

`espaciosDisponibles()`: cuenta las posiciones `null` del arreglo y devuelve cuántas ubicaciones más se pueden registrar.

`ubicacionMasPeligrosa()`: recorre las ubicaciones existentes y devuelve la de mayor riesgo.

Devuelve `null` si el arreglo no tiene registros.

`cantidadPistasRegistradas()`: devuelve `pistas.size()`, que indica cuántas pistas hay registradas.

`pistaMasImportante()`: recorre las pistas y devuelve la de mayor nivel de importancia, o `null` cuando no hay ninguna.

`pistaMasConfiable()`: recorre las pistas y devuelve la de mayor confiabilidad, o `null` si el `ArrayList` está vacío.

`promedioNivelImportancia()`: suma la importancia de las pistas y divide entre su cantidad usando `double`. Si no hay pistas devuelve 0.

`mostrarPistas()`: muestra cada pista y devuelve `true` cuando hay registros. Si el `ArrayList` está vacío devuelve `false`.

`mostrarUbicaciones()`: recorre el arreglo y muestra las ubicaciones distintas de `null`. Devuelve `true` si encontró registros y `false` si no había ninguno.

`cantidadDeEspaciosDeUbicaciones()`: devuelve la cantidad de posiciones vacías. En este código cumple la misma función que `espaciosDisponibles()` y es el método usado por el reporte.

## **Main**

+ main(args: String[]): void {static}

### **Propósito de la clase:**

Es el punto de entrada y se encarga de la interacción con el usuario. Muestra el menú, pide y valida los datos, crea los objetos y utiliza Caso para realizar las operaciones.

### **Propósito de los atributos:**

No tiene atributos de instancia. scanner, casoActual, programaAbierto, validar, opcion y los datos solicitados son variables locales dentro de main().

### **Propósito de los métodos:**

main(args): recibe los argumentos de Java, crea el Scanner y repite el menú hasta elegir la opción 13. Mantiene casoActual, valida entradas con try-catch y llama a Caso. Acepta cancelar o -1 durante las operaciones. Después de nextInt() consume el salto de línea para que las lecturas de texto y las pausas no se salten.

### **Relaciones entre las clases:**

- Main crea y utiliza Caso, Ubicacion y Pista para atender las opciones del menú.
- Caso agrupa de 0 a 5 objetos Ubicacion en el arreglo y de 0 a muchas pistas en ArrayList<Pista>.
- Caso consulta y modifica los objetos mediante sus getters y setters. Los atributos de cada objeto permanecen privados.